



Parul[®]
University

NAAC A++

**FACULTY OF ENGINEERING AND
TECHNOLOGY**

**BACHELOR OF TECHNOLOGY
ARTIFICIAL INTELLIGENCE &
DATA SCIENCE DEPARTMENT**

6th SEMESTER

**Machine Learning Laboratory
(303105354)**

Lab Manual



PARUL UNIVERSITY
FACULTY OF ENGINEERING & TECHNOLOGY
(303105354) Machine Learning Lab
B.Tech. 6th Sem / 3rd Year

CERTIFICATE

This is to certify that student DEASARI PRIYANSH with
Enrollment No. 2403031247004 has
successfully completed his Laboratory experiments in
Machine Learning Laboratory (303105354) from the
department of Artificial Intelligence & Data Science during
the academic year 2025-2026.



Date of Submission: _____

Staff in Charge: _____

Head of Department: _____



Tabel of Content

Sr.No.	Experiment Title	Page No.	Date of Performance	Date of Completion	Sign	Marks
1.	Dealing with Data using Numpy, Pandas, Statistics library					
2.	Data analysis and visualization on Diwali sales dataset					
3.	Implement linear regression and logistic regression					
4.	Implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of classifier, considering a few test data sets.					
5.	Assuming a set of documents that need to be classified, use the naïve Bayesian Classifier model to perform this task.					
6.	Decision tree-based ID3 algorithm.					



Tabel of Content

Sr.No.	Experiment Title	Page No.	Date of Performance	Date of Completion	Sign	Marks
7.	Write a program to implement the K-Nearest Neighbor algorithm to classify the iris data set.					
8.	Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using k-Means algorithm.					
9.	Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set.					
10.	Compare the various supervised learning algorithm by using appropriate dataset. (Linear Regression, Support Vector Machine, Decision Tree)					



Tabel of Content

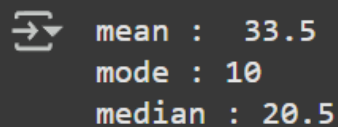
Sr No.	Experiment Title	Page No.	Date of Performance	Date of Completion	Sign	Marks
11.	Compare the various Unsupervised learning algorithm by using the appropriate datasets. (K Means Clustering, K Mode)					
12.	Build an Artificial Neural Network by implementing Backpropagation algorithm and test the same using appropriate data sets					

Practical - 1

Aim:- Dealing with Data using Numpy, Pandas, Statistics library.

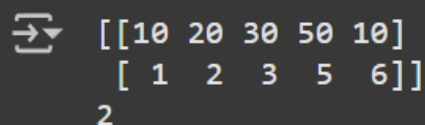
Code:-

```
import statistics as st
a= [10, 20, 50, 10, 21, 90]
mn = st.mean(a)
print("mean : ", mn)
md = st.mode(a)
print("mode :", md)
mdn = st.median(a)
print("median :", mdn)
```



```
mean : 33.5
mode : 10
median : 20.5
```

```
import numpy as np
b = np.array([[10, 20, 30, 50, 10],[1, 2, 3, 5, 6]])
print(b)
print(b.ndim)
```



```
[[10 20 30 50 10]
 [ 1  2  3  5  6]]
2
```

```
b=np.array([[10, 20, 30, 50, 10],[1, 2, 3, 5, 6],[1,1,1,1,1]])
print(b)
print("Dim:",b.ndim)
print("Shape:",b.shape)
print("Size:",b.size)
print("Item Size:",b.itemsize)
print("Data type:",b.dtype)
```

```
→ [[10 20 30 50 10]
    [ 1  2  3  5  6]
    [ 1  1  1  1  1]]
Dim: 2
Shape: (3, 5)
Size: 15
Item Size: 8
Data type: int64
```

```
c=np.empty((5,5))
d=np.arange(0,100,2)
print(d)
e=np.reshape(d,(10,5))
print(e)
f=np.reshape(e,(2,5,5))
print(f)
g = np.full((3,3),5)
print(g)
print("Particular element:",d[10])
```

```
▶ [ 0  2  4  6  8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46
  48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94
  96 98]
→ [[ 0  2  4  6  8]
    [10 12 14 16 18]
    [20 22 24 26 28]
    [30 32 34 36 38]
    [40 42 44 46 48]
    [50 52 54 56 58]
    [60 62 64 66 68]
    [70 72 74 76 78]
    [80 82 84 86 88]
    [90 92 94 96 98]]
[[ 0  2  4  6  8]
 [10 12 14 16 18]
 [20 22 24 26 28]
 [30 32 34 36 38]
 [40 42 44 46 48]]

[[50 52 54 56 58]
 [60 62 64 66 68]
 [70 72 74 76 78]
 [80 82 84 86 88]
 [90 92 94 96 98]]
[[5 5 5]
 [5 5 5]
 [5 5 5]]
Particular element: 20
```

```
print("Element upto 10:",d[:10])  
print("Last 5 element:",d[-10:])  
print("Alternative Numbers:",d[::2])
```

```
Element upto 10: [ 0  2  4  6  8 10 12 14 16 18]  
Last 5 element: [80 82 84 86 88 90 92 94 96 98]  
Alternative Numbers: [ 0  4  8 12 16 20 24 28 32 36 40 44 48 52 56 60 64 68 72 76 80 84 88 92  
96]
```

```
import pandas as pd  
p_dict = {'pid':[1,2,3,4,5], 'value': [10,20,30,40,50],  
'vendor':['V1','V2','V3','V4','V5']}  
print(p_dict)  
df = pd.DataFrame(p_dict)  
df
```

```
{'pid': [1, 2, 3, 4, 5], 'value': [10, 20, 30, 40, 50], 'vendor': ['V1', 'V2', 'V3', 'V4', 'V5']}
```

	pid	value	vendor
0	1	10	V1
1	2	20	V2
2	3	30	V3
3	4	40	V4
4	5	50	V5

Practical 2

Aim :- Data Analysis & Visualization on Diwali Sales Dataset .

Code :-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv('Diwali_sales.csv')
df.columns
df.shape
df.head()
df.info()
df.drop(['Marital_Status', 'Unnamed: 0'],axis=1 ,inplace=True)
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 11251 entries, 0 to 11250
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Unnamed: 0            11251 non-null  int64  
 1   User_ID               11251 non-null  int64  
 2   Cust_name             11251 non-null  object  
 3   Product_ID           11251 non-null  object  
 4   Gender                11251 non-null  object  
 5   Age Group             11251 non-null  object  
 6   Age                   11251 non-null  int64  
 7   Marital_Status        11251 non-null  int64  
 8   State                 11251 non-null  object  
 9   Zone                  11251 non-null  object  
10   Occupation            11251 non-null  object  
11   Product_Category     11251 non-null  object  
12   Orders                11251 non-null  int64  
13   Amount                11239 non-null  float64 
14   age_group             11115 non-null  object  
dtypes: float64(1), int64(5), object(9)
memory usage: 1.3+ MB
```

```
df.columns
pd.isnull(df).sum()
df.dropna(inplace=True)
df.info()
df['Amount']=df['Amount'].astype('int')
df['Amount'].dtypes
df.describe()
df.describe(include='all')
df[['Age','Orders','Amount']].describe()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 11103 entries, 0 to 11250
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID               11103 non-null  int64
1   Cust_name             11103 non-null  object
2   Product_ID           11103 non-null  object
3   Gender                11103 non-null  object
4   Age Group             11103 non-null  object
5   Age                   11103 non-null  int64
6   State                 11103 non-null  object
7   Zone                  11103 non-null  object
8   Occupation            11103 non-null  object
9   Product_Category     11103 non-null  object
10  Orders                11103 non-null  int64
11  Amount                11103 non-null  float64
12  age_group             11103 non-null  object
dtypes: float64(1), int64(3), object(9)
memory usage: 1.2+ MB
```

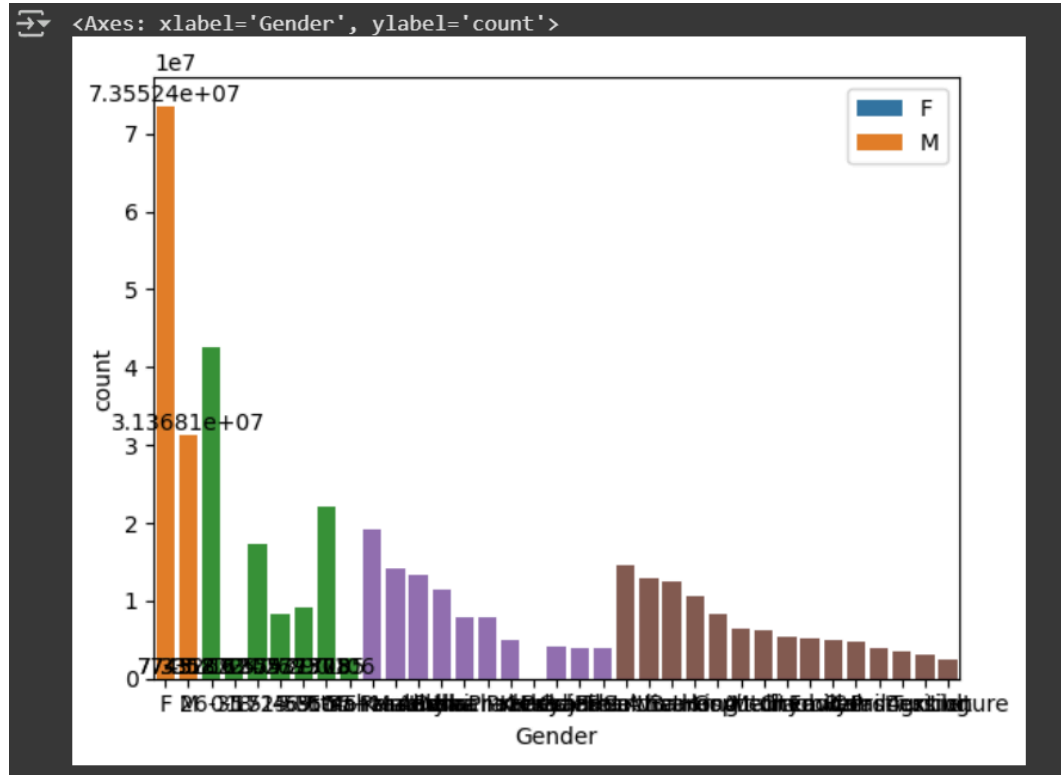
	Age	Orders	Amount
count	11103.000000	11103.000000	11103.000000
mean	34.784473	2.489237	9449.739350
std	11.496090	1.114926	5218.772198
min	12.000000	1.000000	188.000000
25%	27.000000	2.000000	5443.000000
50%	33.000000	2.000000	8109.000000
75%	43.000000	3.000000	12671.000000
max	80.000000	4.000000	23952.000000

```
ax=sns.countplot(x='Gender',data=df)
for bars in ax.containers:
    ax.bar_label(bars)
sales_gen=df.groupby(['Gender'],as_index=False)
['Amount'].sum().sort_values(by='Amount',ascending=False)
sns.barplot(x='Gender',y='Amount',data=sales_gen)
ax=sns.countplot(x='Age Group',hue='Gender', data=df)

for bars in ax.containers:
    ax.bar_label(bars)
sales_age = df.groupby(['Age Group'], as_index=False)
['Amount'].sum().sort_values(by='Amount', ascending=False)
sns.barplot(x='Age Group',y='Amount',data=sales_age)
sales_states=df.groupby(['State'],as_index=False)
['Orders'].sum().sort_values(by='Orders',
ascending=False).head(10)

sns.barplot(x='State',y='Orders',data=sales_states)
sales_state=df.groupby(['State'],as_index=False)
['Amount'].sum().sort_values(by='Amount',
ascending=False).head(10)

sns.barplot(x='State',y='Amount',data=sales_state)
sales_state=df.groupby(['Occupation'],as_index=False)
['Amount'].sum().sort_values(by='Amount',ascending=False)
sns.barplot(x='Occupation',y='Amount',data=sales_state)
```



Practical 3A

Aim :- Implement linear regression and logistic regression.

Code :-

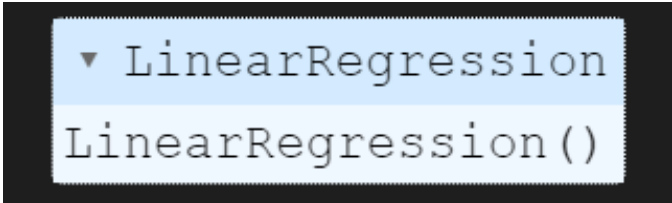
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

datasets = pd.read_csv('C:\\Users\\chayan\\Downloads\\Salary_Data.csv')

X = datasets.iloc[:, :-1].values
Y = datasets.iloc[:, 1].values

from sklearn.model_selection import train_test_split
X_Train, X_Test, Y_Train, Y_Test = train_test_split(X, Y, test_size = 1/3,
random_state = 0)

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_Train, Y_Train)
```



```
plt.scatter(X_Train, Y_Train, color = 'red')
plt.plot(X_Train, regressor.predict(X_Train), color = 'blue')
plt.title('Salary vs Experience (Training Set)')
plt.xlabel('Years of experience')
plt.ylabel('Salary')
plt.show()
```



```
plt.scatter(X_Test, Y_Test, color = 'red')  
plt.plot(X_Train, regressor.predict(X_Train), color = 'blue')  
plt.title('Salary vs Experience (Training Set)')  
plt.xlabel('Years of experience')  
plt.ylabel('Salary')  
plt.show()
```



```
regressor.predict([[7]])
```

```
array([92237.78934589])
```

Practical 3B

Aim :- Implement linear regression and logistic regression.

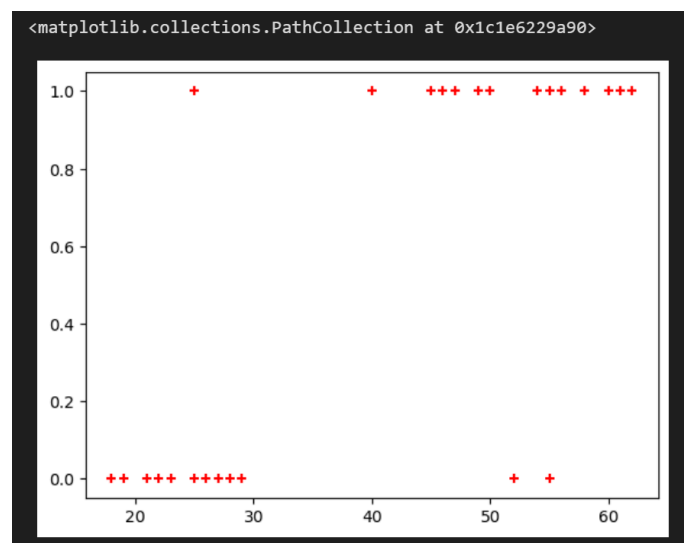
Code :-

```
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline

df = pd.read_csv('C:\\Users\\chayan\\OneDrive\\Desktop\\ML Lab\\Experiment
6\\insurance_data.csv')
df.head()
```

	age	bought_insurance
0	22	0
1	25	0
2	47	1
3	52	0
4	46	1

```
plt.scatter(df.age,df.bought_insurance,marker='+',color='red')
```



```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test =  
train_test_split(df[['age']],df.bought_insurance,train_size=0.8)
```

X_test

age	
3	52
6	55
13	29
9	61
0	22
4	46

X_train

age	
22	40
24	50
15	55
2	47
17	58
19	18
1	25
21	26
16	25
25	54
26	23

8	62
14	49
5	56
10	18
11	28
12	27
7	60
18	19
20	21
23	45

Y_train

```
22    1    26    0
24    1    8     1
15    1   14    1
2     1    5     1
17    1   10    0
19    0   11    0
1     0   12    0
21    0    7     1
16    1   18    0
25    1   20    0
      23    1
Name: bought_insurance, dtype: int64
```

Y_test

```
3     0
6     0
13    0
9     1
0     0
4     1
Name: bought_insurance, dtype: int64
```

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
▼ LogisticRegression
LogisticRegression()
```

```
y_predicted = model.predict(X_test)
model.predict_proba(X_test)
```

```
array([[9.61238370e-04, 9.99038762e-01],
       [3.58342864e-04, 9.99641657e-01],
       [6.50956320e-01, 3.49043680e-01],
       [4.97558461e-05, 9.99950244e-01],
       [9.49166480e-01, 5.08335202e-02],
       [6.88394595e-03, 9.93116054e-01]])
```

model.score(X_test,y_test)

```
0.6666666666666666
```

y_predicted

```
array([-10.16748831])
```

Lets defined sigmoid function now and do the math with hand

model.intercept_

```
array([-10.16748831])
```

Lets defined sigmoid function now and do the math with hand

import math

def sigmoid(x):

 return 1 / (1 + math.exp(-x))

def prediction_function(age):

 z = 0.329* age - 10.16 #

 y = sigmoid(z)

 return y

age = 35

prediction_function(age)

```
0.7949458649106352
```

0.331 is less than 0.5 which means person with 35 age will *not* buy insurance

age = 43

prediction_function(age)

```
0.98178273024916
```

[+ Code](#) [+ Markdown](#)

0.544 is more than 0.5 which means person with 43 will buy the insurance

Practical 4

Aim :- Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

Code :-

```
import pandas as pd
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
data = pd.read_csv('tennisdata.csv - Sheet1.csv')
print("The first 5 values of data is :\n",data.head())
```

```
The first 5 values of data is :
  Unnamed: 0  Outlook  Temperature  Humidity  Windy  PlayTennis
0          0   Sunny         Hot      High   False         No
1          1   Sunny         Hot      High    True         No
2          2  Overcast         Hot      High   False         Yes
3          3   Rainy        Mild      High   False         Yes
4          4   Rainy        Cool     Normal  False         Yes
```

```
X = data.iloc[:, :-1]
print("\nThe First 5 values of train data is\n",X.head())
```

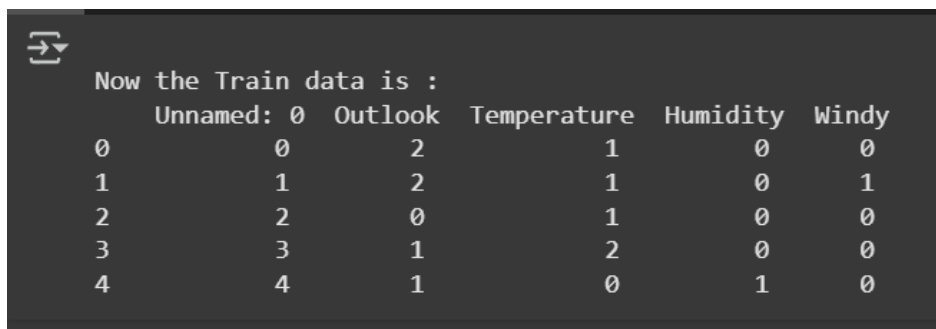
```
The First 5 values of train data is
  Unnamed: 0  Outlook  Temperature  Humidity  Windy
0          0   Sunny         Hot      High   False
1          1   Sunny         Hot      High    True
2          2  Overcast         Hot      High   False
3          3   Rainy        Mild      High   False
4          4   Rainy        Cool     Normal  False
```

```
from sklearn.preprocessing import LabelEncoder
X.columns = X.columns.str.strip()

le_outlook = LabelEncoder()
le_temperature = LabelEncoder()
```

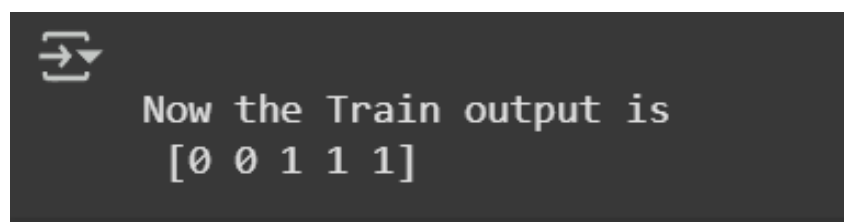
```
le_humidity = LabelEncoder()
le_windy = LabelEncoder()
X['Outlook'] = le_outlook.fit_transform(X['Outlook'])
X['Temperature'] = le_temperature.fit_transform(X['Temperature'])
X['Humidity'] = le_humidity.fit_transform(X['Humidity'])
X['Windy'] = le_windy.fit_transform(X['Windy'])

print("\nNow the Train data is :\n", X.head())
```



```
Now the Train data is :
   Unnamed: 0  Outlook  Temperature  Humidity  Windy
0           0         2            1         0         0
1           1         2            1         0         1
2           2         0            1         0         0
3           3         1            2         0         0
4           4         1            0         1         0
```

```
le_PlayTennis = LabelEncoder()
y = le_PlayTennis.fit_transform(y)
print("\nNow the Train output is\n",y)
```



```
Now the Train output is
[0 0 1 1 1]
```


Practical 5

Aim :- Assuming a set of documents that need to be classified, use the naïve Bayesian Classifier model to perform this task.

Code :-

```
import pandas as pd
msg=pd.read_csv('NaiveText.csv',names=['message','label'])
msg
print('The dimensions of the dataset',msg.shape)
msg
X=msg.message
y=msg.label
X
y
```

The dimensions of the dataset (19, 2)

label

0	label
1	1
2	1
3	1
4	1
5	1
6	0
7	0
8	0
9	0
10	0
11	1

12	0
13	1
14	0
15	1
16	0
17	1
18	0

dtype: object

```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(X,y,test_size=0.20,random_state=0)
```

```
from sklearn.feature_extraction.text import CountVectorizer
count_vect = CountVectorizer()
xtrain_dtm = count_vect.fit_transform(xtrain)
xtest_dtm=count_vect.transform(xtest)
```

```
print('\n The words or Tokens in the text documents \n')
print(count_vect.get_feature_names_out())
```

```
The words or Tokens in the text documents
```

```
['about' 'am' 'amazing' 'an' 'and' 'awesome' 'bad' 'beers' 'best' 'dance'
'do' 'enemy' 'feel' 'fun' 'good' 'great' 'have' 'he' 'holiday' 'is'
'juice' 'like' 'locality' 'love' 'message' 'my' 'not' 'of' 'place'
'restaurant' 'sick' 'stay' 'stuff' 'sworn' 'taste' 'that' 'the' 'these'
'this' 'tired' 'to' 'tomorrow' 'very' 'view' 'we' 'what' 'will' 'work']
```

```
df=pd.DataFrame(xtrain_dtm.toarray(),columns=count_vect.get_feature_names_out())
df
```

```
from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB().fit(xtrain_dtm,ytrain)
predicted = clf.predict(xtest_dtm)
from sklearn import metrics
print("Accuracy = ",metrics.accuracy_score(ytest,predicted))
print('\n Confusion matrix')
print(metrics.confusion_matrix(ytest,predicted))
```

```
⇒ Accuracy = 0.5

Confusion matrix
[[1 2]
 [0 1]]
```

```
CR = metrics.classification_report(ytest,predicted)
print('Classification_report', CR)
```

Classification_report			precision	recall	f1-score	support
	0	1.00	0.33	0.50		3
	1	0.33	1.00	0.50		1
accuracy				0.50		4
macro avg		0.67	0.67	0.50		4
weighted avg		0.83	0.50	0.50		4

Practical 6

Aim:- Write a program to demonstrate the working of the decision tree-based ID3 algorithm.

Code:-

The ID3 (Iterative Dichotomiser 3) algorithm is a foundational decision tree approach in machine learning. It constructs a tree by iteratively selecting the best attribute to split the data based on information gain¹. Here's a high-level overview of how it works:

Selecting the Best Attribute:

- o ID3 calculates entropy and information gain for each attribute in the dataset.
- o Entropy measures the impurity or randomness in the data.
- o The algorithm selects the attribute that results in the most significant information gain when used for splitting the data.

Recursive Tree Construction:

At each internal node, ID3 chooses the attribute with the highest information gain to split the data.

The process continues recursively until a stopping criterion (e.g., minimum subset size or maximum tree depth) is met.

Leaf Nodes:

- o Each leaf node represents a class label or decision.
- o The resulting tree mimics human decision-making processes by splitting data based on different attributes.

To demonstrate ID3

We can implement it in Python. Here's a simplified example using a toy dataset:

```
import numpy as np
```

```
X = np.array([[0, 1], [1, 0], [1, 1], [0, 0]])
y = np.array([1, 0, 1, 0])

def entropy(labels):
    """Calculate entropy given a list of labels."""
    unique_labels, counts = np.unique(labels, return_counts=True)
    probabilities = counts / len(labels)
    return -np.sum(probabilities * np.log2(probabilities + 1e-9))

def information_gain(parent_labels, left_child_labels, right_child_labels):
    """Calculate information gain for a split."""
    parent_entropy = entropy(parent_labels)

    left_weight = len(left_child_labels) / len(parent_labels)
    right_weight = len(right_child_labels) / len(parent_labels)

    child_entropy = (left_weight * entropy(left_child_labels)) + (right_weight *
entropy(right_child_labels))

    return parent_entropy - child_entropy

def id3(X, y):

    if len(np.unique(y)) == 1:
        return y[0]

    if X.shape[1] == 0:
        return np.bincount(y).argmax()
        best_attribute = None
        best_info_gain = -np.inf
        best_splits = None
    for feature in range(X.shape[1]):
        left_child_labels = y[X[:, feature] == 0]
```

```
right_child_labels = y[X[:, feature] == 1]

if len(left_child_labels) == 0 or len(right_child_labels) == 0:
    continue

info_gain = information_gain(y, left_child_labels, right_child_labels)

if info_gain > best_info_gain:
    best_info_gain = info_gain
    best_attribute = feature
    best_splits = (left_child_labels, right_child_labels)

if best_info_gain == -np.inf:
    return np.bincount(y).argmax()

decision_node = {"attribute": best_attribute, "children": {}}

for value in [0, 1]:
    child_indices = np.where(X[:, best_attribute] == value)[0]
    child_X = np.delete(X[child_indices], best_attribute, axis=1)
    child_y = y[child_indices]

    decision_node["children"][value] = id3(child_X, child_y)

return decision_node
tree = id3(X, y)
import pprint
pprint.pprint(tree)
```

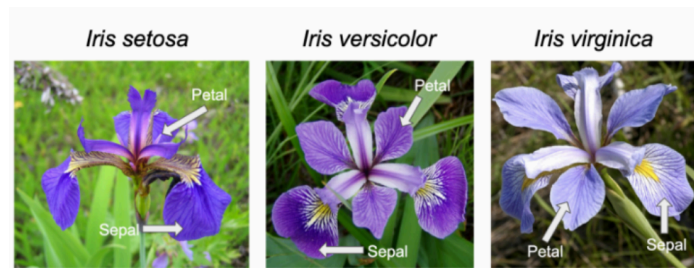
```
⇒ {'attribute': 1, 'children': {0: 0, 1: 1}}
```

Practical 7

Aim:- Write a program to implement the K-Nearest Neighbor algorithm to classify the iris data set

Info:-

What is the K-Nearest Neighbors Algorithm? KNN is one of the most basic yet essential classification algorithms in machine learning. It belongs to the supervised learning domain and finds intense application in pattern recognition, data mining, and intrusion detection.



Pandas is a powerful and widely-used open-source Python library for data manipulation and analysis. It provides data structures and functions specifically designed to make working with structured data (like tables, spreadsheets, and time series) easier and more intuitive.

Key features of Pandas:

Data Structures:

Pandas offers two primary data structures:

Series: A one-dimensional labelled array capable of holding any data type.

DataFrame: A two-dimensional table-like structure with labeled rows and columns, similar to a spreadsheet.

Data Manipulation:

Pandas excels at data manipulation tasks, including:

Reading and writing data from various file formats (e.g., CSV, Excel, SQL databases).

Cleaning and transforming data (e.g., handling missing values, filtering, merging, reshaping).

Aggregating and summarizing data (e.g., calculating statistics, grouping data).

Integration with other libraries: Pandas integrates seamlessly with other popular Python libraries like NumPy, Matplotlib, and Scikit-learn, making it a powerful tool for data analysis and machine learning workflows.

What is Pandas? Pandas is a Python library used for working with data sets. It has functions for analysing, cleaning, exploring, and manipulating data. The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008. Why Use Pandas? Pandas allows us to analyse big data and make conclusions based on statistical theories. Pandas can clean messy data sets and make them readable and relevant. Relevant data is very important in data science.

Scikit-learn, also known as sklearn, is a powerful and widely used machine learning library in Python. It provides a comprehensive set of tools for: Classification: Identifying which category an object belongs to (e.g., spam detection). Regression: Predicting a continuous numerical value (e.g., house price prediction). Clustering: Grouping similar data points together (e.g., customer segmentation). Dimensionality Reduction: Simplifying data by reducing the number of features (e.g., PCA for visualization). Model Selection: Choosing the best model for a given task (e.g., cross-validation, grid search).

Preprocessing: Transforming data into a suitable format for machine learning (e.g., scaling, normalization). Benefits of using scikit-learn: Ease of use: Consistent and intuitive API for various algorithms. Efficiency: Optimized algorithms for fast computation. Versatility: Wide range of algorithms and tools for different tasks. Open source: Actively developed and maintained by a large community. Interoperability: Works seamlessly with other Python scientific libraries like NumPy, Pandas, and Matplotlib.

Code:-

```
import pandas as pd
from sklearn.datasets import load_iris
iris = load_iris()
iris.feature_names
```


	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
100	6.3	3.3	6.0	2.5	2
101	5.8	2.7	5.1	1.9	2
102	7.1	3.0	5.9	2.1	2
103	6.3	2.9	5.6	1.8	2
104	6.5	3.0	5.8	2.2	2

```
df['flower_name'] = df.target.apply(lambda x: iris.target_names[x])
df.head()
```

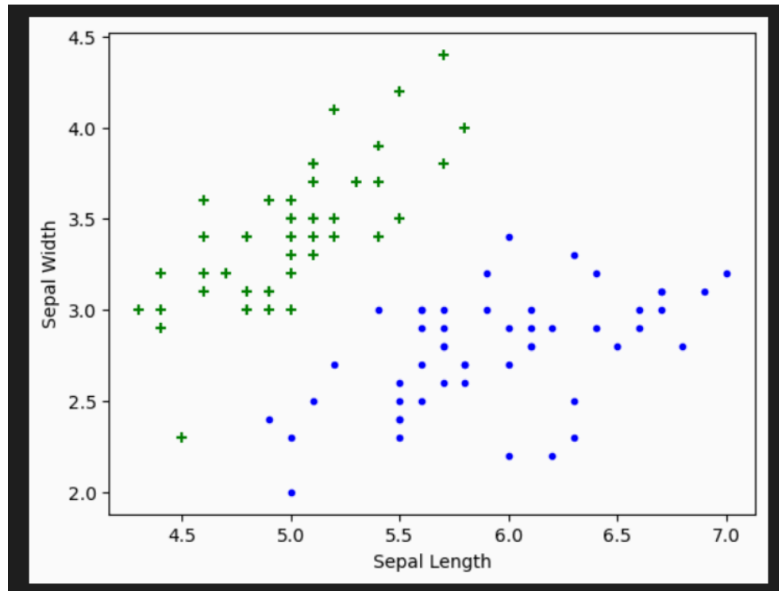
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	flower_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

```
df[45:55]
```

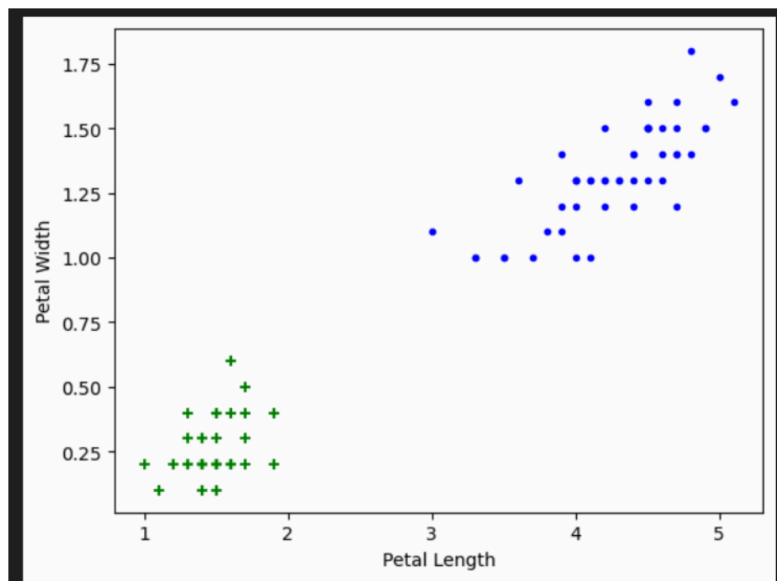
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	flower_name
45	4.8	3.0	1.4	0.3	0	setosa
46	5.1	3.8	1.6	0.2	0	setosa
47	4.6	3.2	1.4	0.2	0	setosa
48	5.3	3.7	1.5	0.2	0	setosa
49	5.0	3.3	1.4	0.2	0	setosa
50	7.0	3.2	4.7	1.4	1	versicolor
51	6.4	3.2	4.5	1.5	1	versicolor
52	6.9	3.1	4.9	1.5	1	versicolor
53	5.5	2.3	4.0	1.3	1	versicolor
54	6.5	2.8	4.6	1.5	1	versicolor

```
df0 = df[:50]
df1 = df[50:100]
df2 = df[100:]
import matplotlib.pyplot as plt %matplotlib inline
```

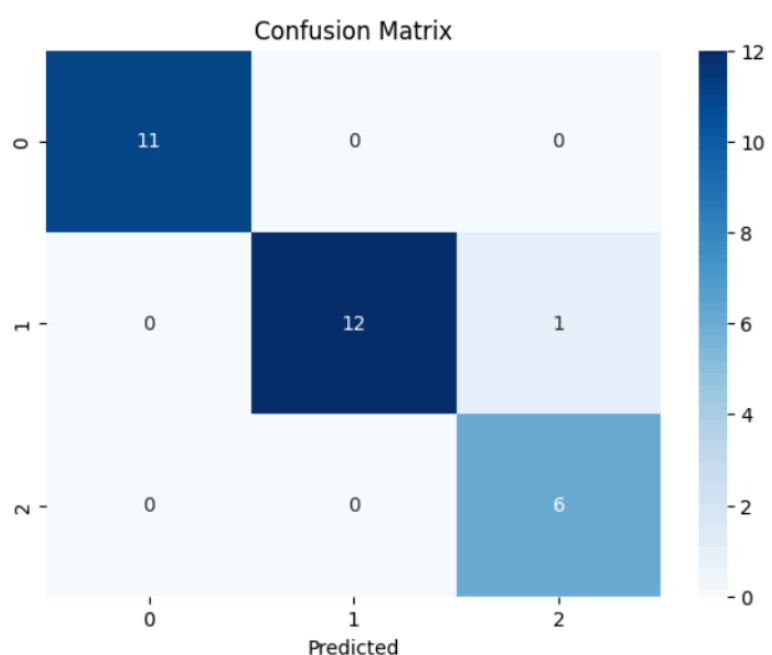
```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.scatter(df0['sepal length (cm)'],
df0['sepal width (cm)',color="green",marker='+')
plt.scatter(df1['sepal length (cm)'], df1['sepal width (cm)'],
color="blue",marker='.')
```



```
plt.xlabel('Petal Length')  
plt.ylabel('Petal Width')  
plt.scatter(df0['petal length (cm)'], df0['petal width (cm)'],color="green",marker='+')  
plt.scatter(df1['petal length (cm)'], df1['petal width (cm)'],color="blue",marker='.')
```



```
from sklearn.model_selection import train_test_split
X = df.drop(['target','flower_name'], axis='columns')
y = df.target
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=1)
len(X_train)
len(X_test)
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=10) knn.fit(X_train, y_train)
knn.score(X_test, y_test) knn.predict([[4.8,3.0,1.5,0.3]])
from sklearn.metrics import confusion_matrix
y_pred = knn.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
cm
%matplotlib
import matplotlib.pyplot as plt
import seaborn as sn
plt.figure(figsize=(7,5)) sn.heatmap(cm, annot=True)
plt.xlabel('Predicted') plt.ylabel('Truth')
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```



Practical 8

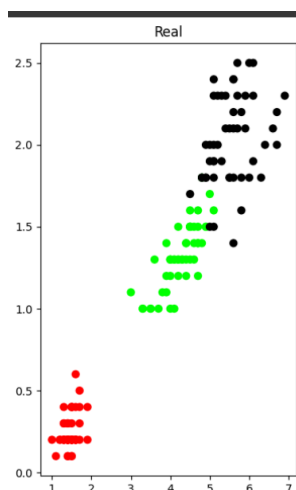
AIM:- Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using k-Means algorithm.

Code:-

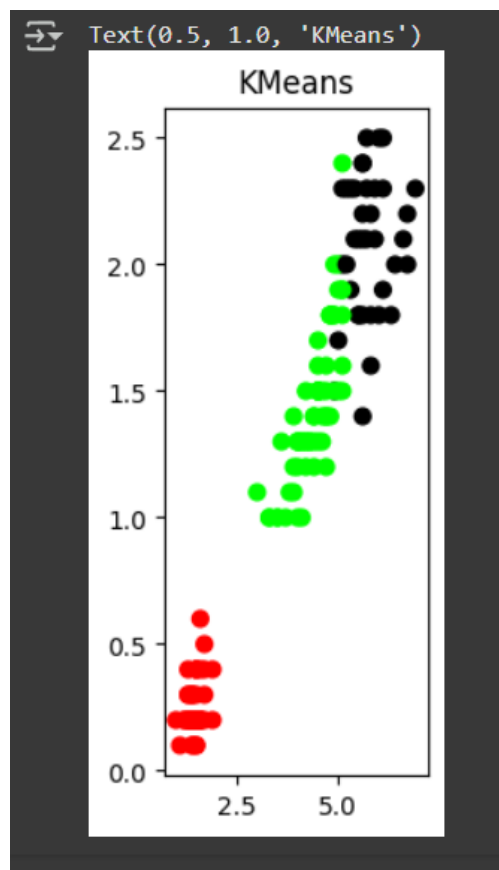
```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
import pandas as pd import numpy as np
import matplotlib.pyplot as plt
dataset=load_iris()
X=pd.DataFrame(dataset.data)
X.columns=
['Sepal_Length','Sepal_Width','Petal_Length','Petal_Width']
y=pd.DataFrame(dataset.target)
y.columns=['Targets']
plt.figure(figsize=(14,7))
colormap=np.array(['red','lime','black'])
```

<Figure size 1400x700 with 0 Axes>

```
plt.subplot(1,3,1)
plt.scatter(X.Petal_Length, X.Petal_Width, c= colormap[y.Targets], s= 40)
plt.title('Real')
plt.show()
```



```
# K-PLOT  
plt.subplot(1,3,2)  
model=KMeans(n_clusters=3)  
  
model.fit(X)  
predY=np.choose(model.labels_,[0,1,2]).astype(np.int64)  
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[predY],s=40)  
plt.title('KMeans')
```



Practical 9

AIM:- Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set.

Code:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
import seaborn as sns
warnings.filterwarnings('ignore')
data=pd.read_csv('/content/heart_statlog_cleveland_hungary_final.csv')
data.shape
```

 (1190, 12)

data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1190 entries, 0 to 1189
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   age                   1190 non-null  int64
 1   sex                   1190 non-null  int64
 2   chest pain type       1190 non-null  int64
 3   resting bp s         1190 non-null  int64
 4   cholesterol           1190 non-null  int64
 5   fasting blood sugar   1190 non-null  int64
 6   resting ecg          1190 non-null  int64
 7   max heart rate        1190 non-null  int64
 8   exercise angina       1190 non-null  int64
 9   oldpeak              1190 non-null  float64
10   ST slope              1190 non-null  int64
11   target               1190 non-null  int64
dtypes: float64(1), int64(11)
memory usage: 111.7 KB
```

data.head()

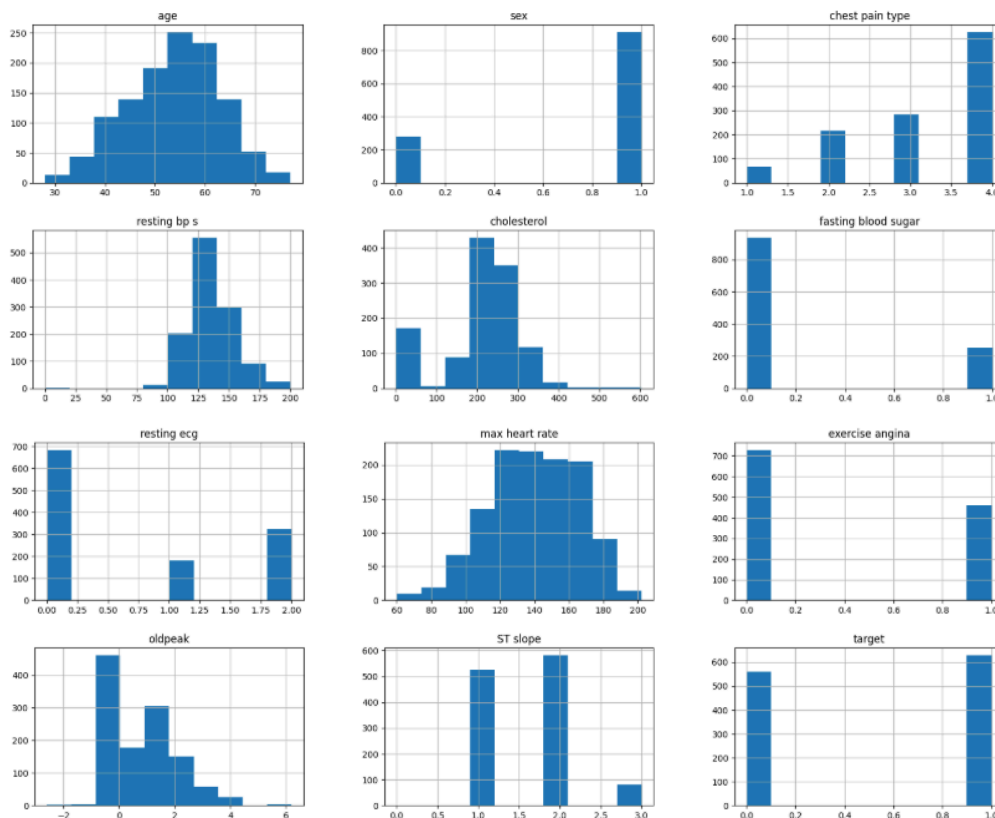
	age	sex	chest pain type	resting bp s	cholesterol	fasting blood sugar	resting ecg	max heart rate	exercise angina	oldpeak	ST slope	target
0	40	1	2	140	289	0	0	172	0	0.0	1	0
1	49	0	3	160	180	0	0	156	0	1.0	2	1
2	37	1	2	130	283	0	1	98	0	0.0	1	0
3	48	0	4	138	214	0	0	108	1	1.5	2	1
4	54	1	3	150	195	0	0	122	0	0.0	1	0

data.describe()

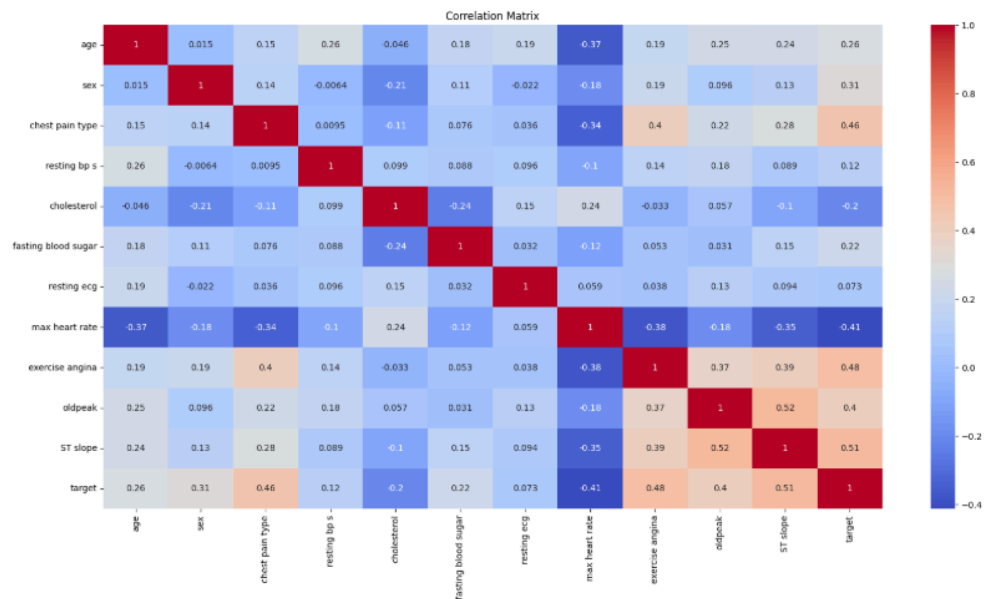
	age	sex	chest pain type	resting bp s	cholesterol	fasting blood sugar	resting ecg	max heart rate	exercise angina	oldpeak	ST slope	target
count	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000	1190.000000
mean	53.720168	0.763866	3.232773	132.153782	210.363866	0.213445	0.698319	139.732773	0.387395	0.922773	1.624370	0.528571
std	9.358203	0.424884	0.935480	18.368823	101.420489	0.409912	0.870359	25.517636	0.487360	1.086337	0.610459	0.499393
min	28.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	60.000000	0.000000	-2.600000	0.000000	0.000000
25%	47.000000	1.000000	3.000000	120.000000	188.000000	0.000000	0.000000	121.000000	0.000000	0.000000	1.000000	0.000000
50%	54.000000	1.000000	4.000000	130.000000	229.000000	0.000000	0.000000	140.500000	0.000000	0.600000	2.000000	1.000000
75%	60.000000	1.000000	4.000000	140.000000	269.750000	0.000000	2.000000	160.000000	1.000000	1.600000	2.000000	1.000000
max	77.000000	1.000000	4.000000	200.000000	603.000000	1.000000	2.000000	202.000000	1.000000	6.200000	3.000000	1.000000

```
data.hist(figsize=(20,16))
```

```
array([[<Axes: title={'center': 'age'}>, <Axes: title={'center': 'sex'}>,
<Axes: title={'center': 'chest pain type'}>],
<Axes: title={'center': 'resting bp s'}>,
<Axes: title={'center': 'cholesterol'}>,
<Axes: title={'center': 'fasting blood sugar'}>],
<Axes: title={'center': 'resting ecg'}>,
<Axes: title={'center': 'max heart rate'}>,
<Axes: title={'center': 'exercise angina'}>],
<Axes: title={'center': 'oldpeak'}>,
<Axes: title={'center': 'ST slope'}>,
<Axes: title={'center': 'target'}>]], dtype=object)
```



```
corr_matrix=data.corr()
plt.figure(figsize=(20,10))
sns.heatmap(corr_matrix,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

```
data.isnull().sum()
```

```
[1]
age      0
sex      0
chest pain type  0
resting bp s  0
cholesterol  0
fasting blood sugar  0
resting ecg  0
max heart rate  0
exercise angina  0
oldpeak      0
ST slope     0
target       0
dtype: int64
```

```
data.duplicated().sum()
```

```
272
```

```
data["target"].value_counts()
```

```
count
target
1      629
0      561
dtype: int64
```

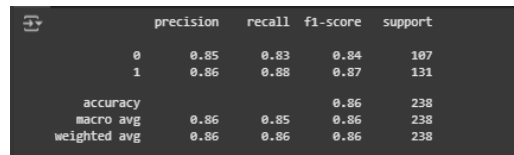
```
x_norm=data.drop(['target'],axis=1)
```

```
y=data['target']
```

```

from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
x=scaler.fit_transform(x_norm)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report nb=GaussianNB()
nb.fit(x_train,y_train)
y_pred=nb.predict(x_test)
print(classification_report(y_test,y_pred))

```

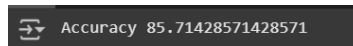


	precision	recall	f1-score	support
0	0.85	0.83	0.84	107
1	0.86	0.88	0.87	131
accuracy			0.86	238
macro avg	0.86	0.85	0.86	238
weighted avg	0.86	0.86	0.86	238

```

from sklearn.metrics import accuracy_score,confusion_matrix
acc=accuracy_score(y_test,y_pred)*100
print("Accuracy",acc)

```

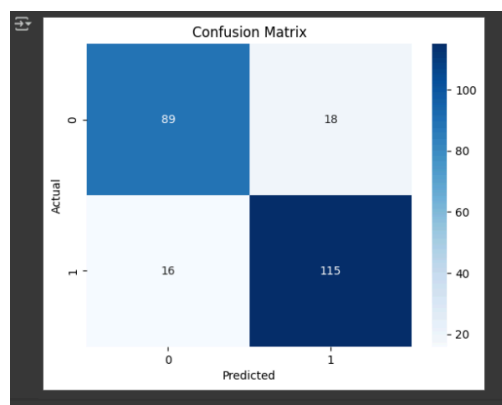


Accuracy 85.71428571428571

```

nb_cm=confusion_matrix (y_test,y_pred)
sns.heatmap(nb_cm,annot=True,fmt='d',cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

```



Practical 10

AIM:- Compare the various supervised learning algorithm by using appropriate dataset. (Linear Regression, Support Vector Machine, Decision Tree)

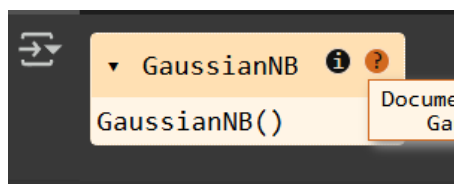
Code:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
import seaborn as sns
warnings.filterwarnings('ignore')

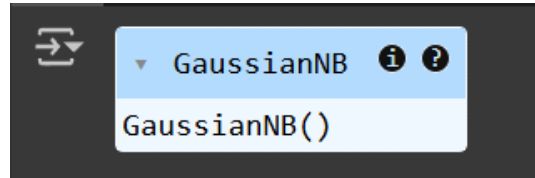
data=pd.read_csv('/content/heart_statlog_cleveland_hungary_final.csv')

data["target"].value_counts()

x_norm=data.drop(['target'],axis=1)
y=data['target']
#scale data
from sklearn.preprocessing import StandardScaler scaler=StandardScaler()
x=scaler.fit_transform(x_norm)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
# Gaussian Naïve Bayes
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report
nb=GaussianNB() nb
```

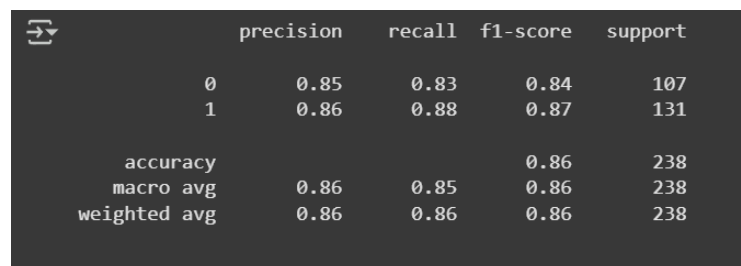


```
nb.fit(x_train,y_train)
```



```
y_pred=nb.predict(x_test)
```

```
print(classification_report(y_test,y_pred))
```



	precision	recall	f1-score	support
0	0.85	0.83	0.84	107
1	0.86	0.88	0.87	131
accuracy			0.86	238
macro avg	0.86	0.85	0.86	238
weighted avg	0.86	0.86	0.86	238

```
from sklearn.metrics import accuracy_score,confusion_matrix
```

```
acc=accuracy_score(y_test,y_pred)*100
```

```
print("Accuracy=.acc")
```

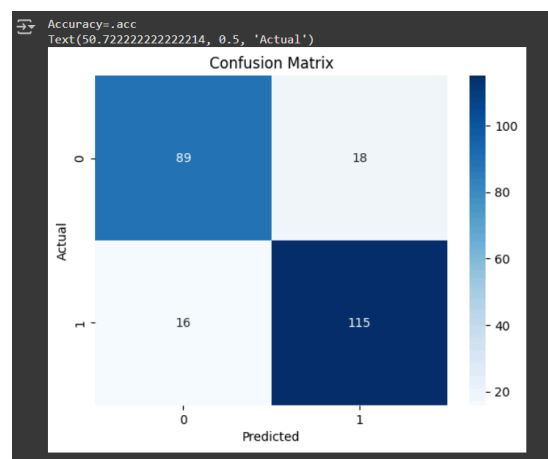
```
nb_cm=confusion_matrix(y_test,y_pred)
```

```
sns.heatmap(nb_cm,annot=True,fmt='d',cmap='Blues')
```

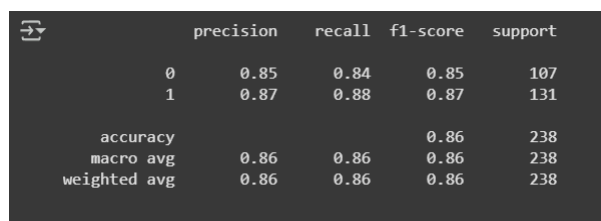
```
plt.title('Confusion Matrix')
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

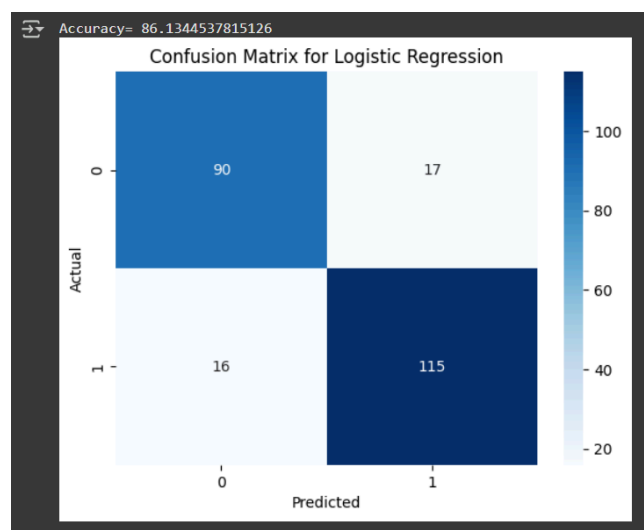


```
#Logistic Regression
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(x_train, y_train)
y_pred_lr = lr.predict(x_test)
print(classification_report(y_test, y_pred_lr))
```



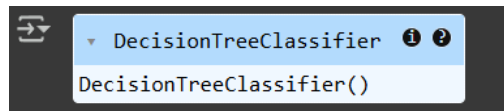
	precision	recall	f1-score	support
0	0.85	0.84	0.85	107
1	0.87	0.88	0.87	131
accuracy			0.86	238
macro avg	0.86	0.86	0.86	238
weighted avg	0.86	0.86	0.86	238

```
acc_lr = accuracy_score(y_test, y_pred_lr) * 100
print("Accuracy=",acc_lr)
lr_cm = confusion_matrix(y_test, y_pred_lr)
sns.heatmap(lr_cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix for Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



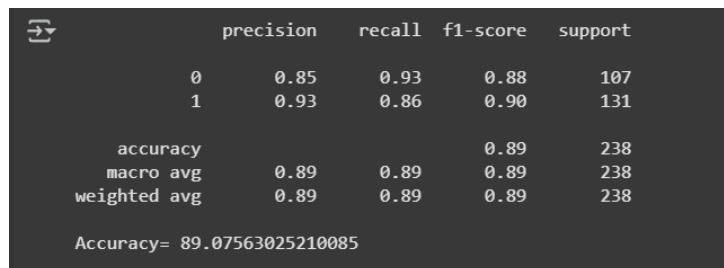
```
# Decision Tree classifier
```

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(x_train, y_train)
```



```
# Make predictions on the test set
y_pred_dt = dt.predict(x_test)
# Evaluate the model
print(classification_report(y_test, y_pred_dt))

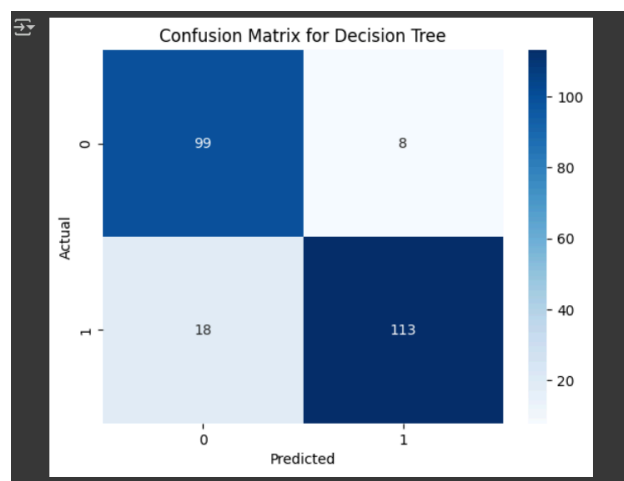
acc_dt = accuracy_score(y_test, y_pred_dt) * 100
print("Accuracy=", acc_dt)
```



	precision	recall	f1-score	support
0	0.85	0.93	0.88	107
1	0.93	0.86	0.90	131
accuracy			0.89	238
macro avg	0.89	0.89	0.89	238
weighted avg	0.89	0.89	0.89	238

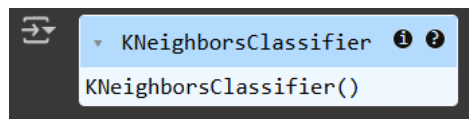
Accuracy= 89.07563025210085

```
dt_cm = confusion_matrix(y_test, y_pred_dt)
sns.heatmap(dt_cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix for Decision Tree')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



#KNN classifier

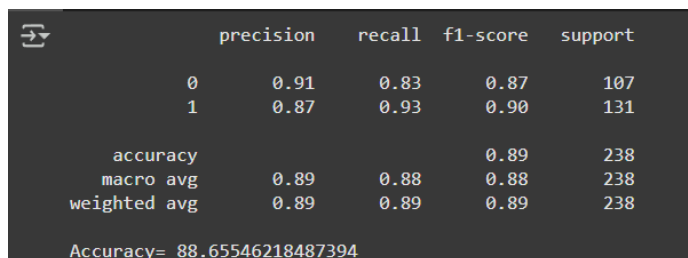
```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5) # You can adjust the number of
neighbors
knn.fit(x_train, y_train)
```



```
# Make predictions on the test set
y_pred_knn = knn.predict(x_test)
```

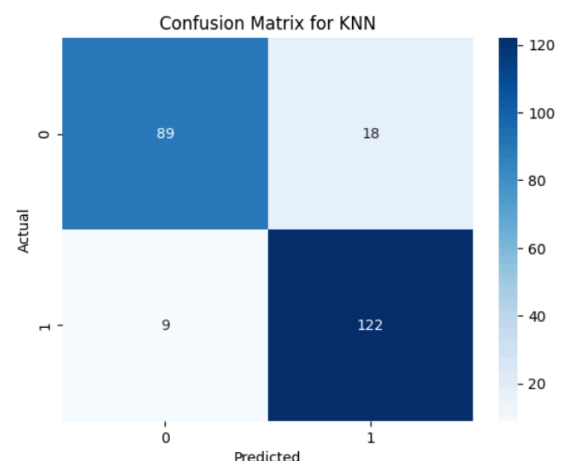
```
# Evaluate the model
print(classification_report(y_test, y_pred_knn))
```

```
acc_knn = accuracy_score(y_test, y_pred_knn) * 100
print("Accuracy=",acc_knn)
knn_cm = confusion_matrix(y_test, y_pred_knn)
sns.heatmap(knn_cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix for KNN')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



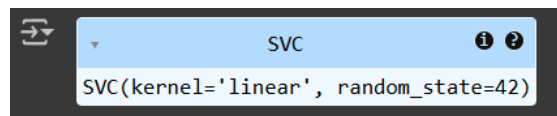
	precision	recall	f1-score	support
0	0.91	0.83	0.87	107
1	0.87	0.93	0.90	131
accuracy			0.89	238
macro avg	0.89	0.88	0.88	238
weighted avg	0.89	0.89	0.89	238

Accuracy= 88.65546218487394



#SVM classifier

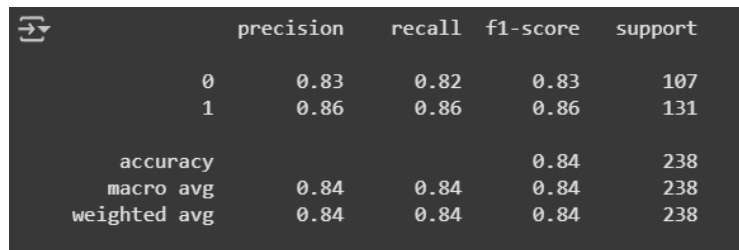
```
from sklearn.svm import SVC
svm = SVC(kernel='linear', random_state=42) # You can change the kernel (e.g.,
'rbf', 'poly')
svm.fit(x_train, y_train)
```



```
SVC(kernel='linear', random_state=42)
```

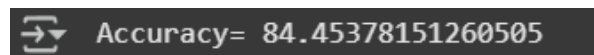
```
# Make predictions on the test set
y_pred_svm = svm.predict(x_test)
```

```
# Evaluate the model
print(classification_report(y_test, y_pred_svm))
```



	precision	recall	f1-score	support
0	0.83	0.82	0.83	107
1	0.86	0.86	0.86	131
accuracy			0.84	238
macro avg	0.84	0.84	0.84	238
weighted avg	0.84	0.84	0.84	238

```
acc_svm = accuracy_score(y_test, y_pred_svm) * 100
print("Accuracy=", acc_svm)
```

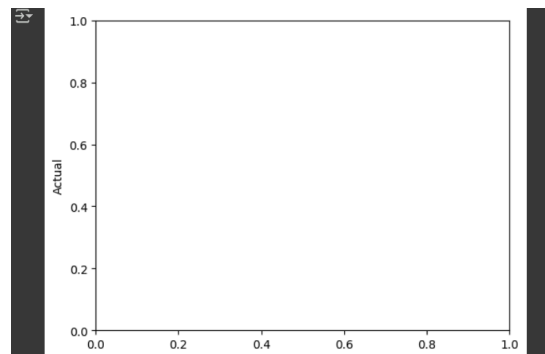


```
Accuracy= 84.45378151260505
```

```
svm_cm = confusion_matrix(y_test, y_pred_svm)
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix for SVM')
plt.xlabel('Predicted')
```



```
best_model = max(model_accuracies, key=model_accuracies.get)
best_accuracy = model_accuracies[best_model]
print(f"The best performing model is {best_model} with an accuracy of
{best_accuracy:.2f}%")
plt.ylabel('Actual')
plt.show()
```



```
model_accuracies = {
    'Naive Bayes': acc,
    'Logistic Regression': acc_lr,
    'Decision Tree': acc_dt,
    'KNN': acc_knn,
    'SVM': acc_svm
}
best_model = max(model_accuracies, key=model_accuracies.get)
best_accuracy = model_accuracies[best_model]
print(f"The best performing model is {best_model} with an accuracy of
{best_accuracy:.2f}%")
```

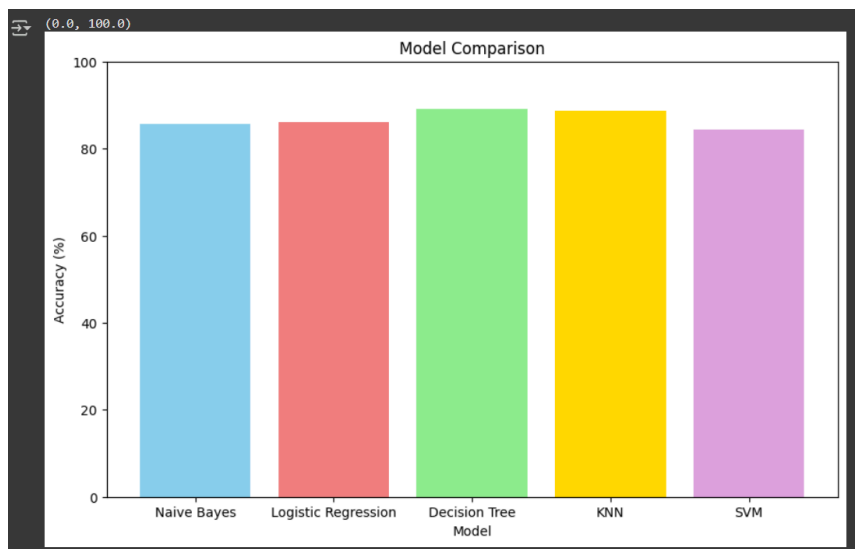
```
→ The best performing model is Decision Tree with an accuracy of 89.08%
```

```
model_data = {
    'Model': ['Naive Bayes', 'Logistic Regression', 'Decision Tree', 'KNN', 'SVM'],
    'Accuracy': [acc, acc_lr, acc_dt, acc_knn, acc_svm]
}
model_comparison = pd.DataFrame(model_data) model_comparison
```



	Model	Accuracy
0	Naive Bayes	85.714286
1	Logistic Regression	86.134454
2	Decision Tree	89.075630
3	KNN	88.655462
4	SVM	84.453782

```
colors = ['skyblue', 'lightcoral', 'lightgreen', 'gold', 'plum']  
plt.figure(figsize=(10, 6))  
plt.bar(model_comparison['Model'], model_comparison['Accuracy'], color=colors)  
plt.xlabel("Model")  
plt.ylabel("Accuracy (%)")  
plt.title("Model Comparison")  
plt.ylim(0, 100) # Set y-axis limit to 100%
```



Practical 11

AIM:- Compare the various Unsupervised learning algorithm by using the appropriate datasets. (K Means Clustering, K Mode)

Code:-

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from scipy.optimize import linear_sum_assignment
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Load the iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Apply K-Means clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X)

# Function to match predicted cluster labels to true labels
def cluster_labels_to_true_labels(y_true, y_pred):
    D = np.zeros((np.max(y_pred) + 1, np.max(y_true) + 1), dtype=np.int32)
    for i in range(y_pred.size):
        D[y_pred[i], y_true[i]] += 1
    row_ind, col_ind = linear_sum_assignment(D.max() - D)
    return col_ind

# Function to compute evaluation metrics
def compute_metrics(y_true, y_pred):
```

```
y_pred_matched = cluster_labels_to_true_labels(y_true, y_pred)[y_pred]
accuracy = accuracy_score(y_true, y_pred_matched)
precision = precision_score(y_true, y_pred_matched, average='weighted')
recall = recall_score(y_true, y_pred_matched, average='weighted')
f1 = f1_score(y_true, y_pred_matched, average='weighted')
return accuracy, precision, recall, f1
```

```
# Compute metrics for K-Means clustering
```

```
kmeans_metrics = compute_metrics(y, kmeans_labels)
```

```
# Plot the results
```

```
df = pd.DataFrame(X, columns=iris.feature_names)
```

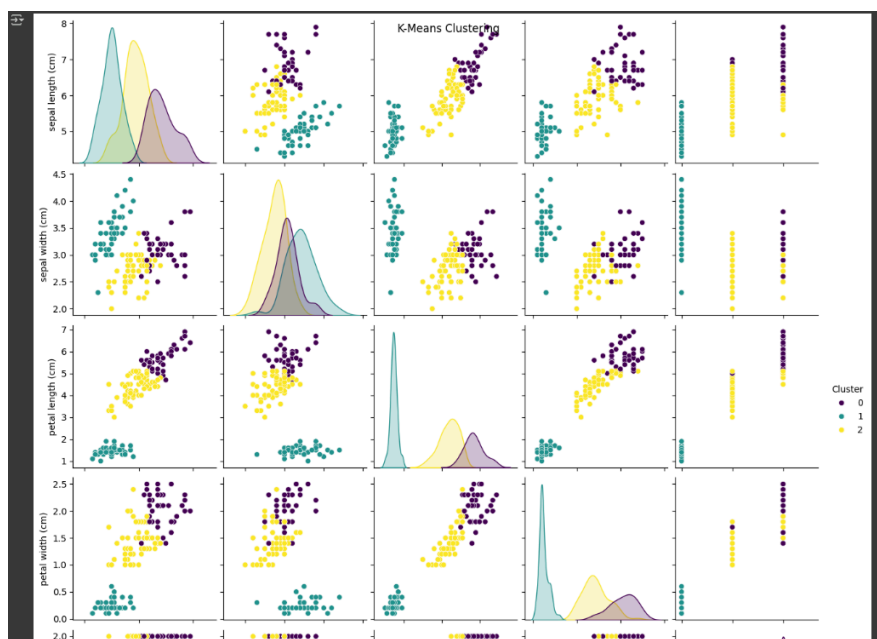
```
df['Cluster'] = kmeans_labels
```

```
df['True Label'] = y
```

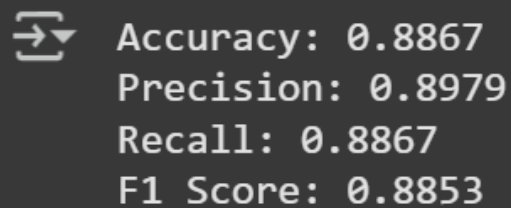
```
sns.pairplot(df, hue='Cluster', palette='viridis')
```

```
plt.suptitle('K-Means Clustering')
```

```
plt.show()
```



```
# Print evaluation metrics  
print(f"Accuracy: {kmeans_metrics[0]:.4f}")  
print(f"Precision: {kmeans_metrics[1]:.4f}")  
print(f"Recall: {kmeans_metrics[2]:.4f}")  
print(f"F1 Score: {kmeans_metrics[3]:.4f}")
```

The image shows a terminal window with a dark background and light gray text. It displays the output of the Python code: Accuracy: 0.8867, Precision: 0.8979, Recall: 0.8867, and F1 Score: 0.8853. A cursor icon is visible at the start of the first line.

```
⇒ Accuracy: 0.8867  
Precision: 0.8979  
Recall: 0.8867  
F1 Score: 0.8853
```

Practical 12

AIM:- Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets

Code:-

```
import tensorflow as tf
from tensorflow.keras import datasets

(x_train, y_train), (x_test, y_test) = datasets.boston_housing.load_data()

print('Training set: ', x_train.shape, y_train.shape)
print('Test set: ', x_test.shape, y_test.shape)

⇒ Training set: (404, 13) (404,)
   Test set: (102, 13) (102,)

x_train[:5]

⇒ array([15.2, 42.3, 50. , 21.1, 17.7])

x_train[0]

⇒ array([ 1.23247,  0.      ,  8.14   ,  0.      ,  0.538   ,  6.142   ,
          91.7    ,  3.9769 ,  4.      , 307.    ,  21.     , 396.9   ,
          18.72   ])
```

Build ANN Arch for Regression

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import plot_model

model_reg = Sequential([
    Input((13,)),
    Dense(1, activation='linear')
])

# model_reg.summary()
# plot_model(model_reg, show_shapes=True, show_layer_activations=True)
```

```
# Model Training
model_reg.compile(optimizer='sgd', loss='msle', metrics=['accuracy', 'msle'])
model_reg.fit(x_train, y_train, epochs=10)
```

→ Epoch 1/10
13/13 ————— 1s 2ms/step - accuracy: 0.0000e+00 - loss: 6.4965 - msle
Epoch 2/10
13/13 ————— 0s 1ms/step - accuracy: 0.0000e+00 - loss: 9.5771 - msle
Epoch 3/10
13/13 ————— 0s 1ms/step - accuracy: 0.0000e+00 - loss: 9.7181 - msle
Epoch 4/10
13/13 ————— 0s 1ms/step - accuracy: 0.0000e+00 - loss: 9.5237 - msle
Epoch 5/10
13/13 ————— 0s 2ms/step - accuracy: 0.0000e+00 - loss: 9.4521 - msle
Epoch 6/10
13/13 ————— 0s 1ms/step - accuracy: 0.0000e+00 - loss: 9.6827 - msle
Epoch 7/10
13/13 ————— 0s 2ms/step - accuracy: 0.0000e+00 - loss: 9.5076 - msle
Epoch 8/10
13/13 ————— 0s 1ms/step - accuracy: 0.0000e+00 - loss: 9.6568 - msle
Epoch 9/10
13/13 ————— 0s 2ms/step - accuracy: 0.0000e+00 - loss: 9.7762 - msle
Epoch 10/10
13/13 ————— 0s 2ms/step - accuracy: 0.0000e+00 - loss: 9.7529 - msle
<keras.src.callbacks.history.History at 0x7d5dd8624940>

Evaluate the model

```
model_reg.evaluate(x_test, y_test)
```

→ 4/4 ————— 0s 3ms/step - accuracy: 0.0000e+00 - loss: 9.6913 - msle: 9.
[9.806747436523438, 0.0, 9.806747436523438]

Build ANN Arch for Regression

```
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import Dense, Input
```

```
from tensorflow.keras.utils import plot_model
```

```
model_reg_1 = Sequential()
```

```
model_reg_1.add(Dense(50, activation='relu', input_shape=(x_train.shape[1], )))
```

```
model_reg_1.add(Dense(25, activation='relu'))
```

```
model_reg_1.add(Dense(12, activation='relu'))
```

```
model_reg_1.add(Dense(1, activation='linear'))
```



```
# model_reg_1.summary()
# plot_model(model_reg_1, show_shapes=True, show_layer_activations=True)

# Model Training
model_reg_1.compile(optimizer='adam', loss='msle', metrics=['accuracy', 'msle'])
model_reg_1.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test), batch_size

➡ Epoch 1/10
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
7/7 ————— 2s 41ms/step - accuracy: 0.0000e+00 - loss: 0.3993 - msle: 0.
Epoch 2/10
7/7 ————— 0s 6ms/step - accuracy: 0.0000e+00 - loss: 0.1851 - msle: 0.
Epoch 3/10
7/7 ————— 0s 10ms/step - accuracy: 0.0000e+00 - loss: 0.1372 - msle: 0.
Epoch 4/10
7/7 ————— 0s 9ms/step - accuracy: 0.0000e+00 - loss: 0.1228 - msle: 0.
Epoch 5/10
7/7 ————— 0s 6ms/step - accuracy: 0.0000e+00 - loss: 0.1111 - msle: 0.
Epoch 6/10
7/7 ————— 0s 9ms/step - accuracy: 0.0000e+00 - loss: 0.0861 - msle: 0.
Epoch 7/10
7/7 ————— 0s 9ms/step - accuracy: 0.0000e+00 - loss: 0.0781 - msle: 0.
Epoch 8/10
7/7 ————— 0s 9ms/step - accuracy: 0.0000e+00 - loss: 0.0768 - msle: 0.
Epoch 9/10
7/7 ————— 0s 11ms/step - accuracy: 0.0000e+00 - loss: 0.0980 - msle: 0.
Epoch 10/10
7/7 ————— 0s 6ms/step - accuracy: 0.0000e+00 - loss: 0.0816 - msle: 0.
<keras.src.callbacks.history.History at 0x7d5dd844fdc0>
```

```
# ANN for binary classification
```

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

bank_ds = pd.read_csv('/content/Bank(1 ).csv', delimiter=';')

le = LabelEncoder()

for col in ['job', 'marital', 'default', 'education', 'housing', 'loan', 'contact', 'mont
    bank_ds[col] = le.fit_transform(bank_ds[col])
bank_ds.head()
```

```
➡ age job marital education default balance housing loan contact day month
```


0	30	10	1	0	0	1787	0	0	0	19	10
1	33	7	1	1	0	4789	1	1	0	11	8
2	35	4	2	2	0	1350	1	0	0	16	0
3	30	4	1	2	0	1476	1	1	2	3	6
4	59	1	1	1	0	0	1	0	2	5	8

Next
steps:

Generate code
with bank_ds



View recommended
plots

New interactive
sheet

```
x = bank_ds.iloc[:, :-1].values
y = bank_ds.iloc[:, -1].values
```

```
# import numpy as np
```

```
# x.shape
```

```
# for i in range(x.shape[1]):
#     x[:, i] = x[:, i] / np.max(x[:, i])
```

```
from sklearn.preprocessing import StandardScaler
```

```
x_std = StandardScaler().fit_transform(x)
```

```
x_std[0]
```

```
array([[-1.05626965,  1.71680374, -0.24642938, -1.64475535, -0.1307588 ,
         0.12107186, -1.14205138, -0.42475611, -0.72364152,  0.37405206,
         1.48541444, -0.7118608 , -0.57682947, -0.4072183 , -0.32041282,
         0.44441328])
```

```
1      0.38      0.29      0.33      521
```

```
accuracy      0.86      4521
```

```
macro avg    0.64      0.61      0.63      4521
```

```
weighted avg  0.85      0.86      0.86      4521
```

```
from tensorflow.keras import losses
from tensorflow.keras import optimizers
```

```
# ANN for multiclass class
```

```
from sklearn import datasets
from tensorflow.keras.utils import to_categorical

iris_ds = datasets.load_iris()

x = iris_ds.data
y = iris_ds.target

y_cat = to_categorical(y, num_classes=3) #one-hot encoding

model_mc = Sequential()
model_mc.add(Dense(10, activation='relu', input_shape=(4,)))
model_mc.add(Dense(5, activation='relu'))
model_mc.add(Dense(3, activation='softmax')) #multiclass classification take softmax

model_mc.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model_mc.fit(x, y_cat, epochs=4, batch_size=64, validation_split=0.2)
```

```
Epoch 1/4
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2/2 ----- 1s 219ms/step - accuracy: 0.4236 - loss: 1.1951 - val_accuracy: 0.4236
Epoch 2/4
2/2 ----- 0s 33ms/step - accuracy: 0.4132 - loss: 1.1791 - val_accuracy: 0.4132
Epoch 3/4
2/2 ----- 0s 28ms/step - accuracy: 0.3924 - loss: 1.1723 - val_accuracy: 0.3924
Epoch 4/4
2/2 ----- 0s 27ms/step - accuracy: 0.4080 - loss: 1.1435 - val_accuracy: 0.4080
<keras.src.callbacks.history.History at 0x7d5dc99d5870>
```

```
import numpy as np

x.shape
np.unique(y)

array([0, 1, 2])

y_pred_proba = model_mc.predict(x)

y_pred = []
for proba in y_pred_proba:
```

```
y_pred.append(np.argmax(proba))
```

5/5 ————— 0s 10ms/step

```
print(classification_report(y, y_pred))
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	50
1	0.33	1.00	0.50	50
2	0.00	0.00	0.00	50
accuracy			0.33	150
macro avg	0.11	0.33	0.17	150
weighted avg	0.11	0.33	0.17	150